

CAPÍTULO 4

Ejemplos de Aplicación de Hilos



```
$ gcc -o primos primos.c -lpthread  
$ ./primos  
Primo #5000 = 48611  
Primo #6000 = 59359  
Primo #7000 = 70657
```

Contenido

1

Cómputo paralelo

Varios primos a la vez, repartidos entre hilos

\$4.1

2

Cancelación segura

Cancelar una tarea sin fugas de memoria

\$4.2

3

Datos específicos de hilo

Un archivo de log distinto para cada hilo

\$4.3

4

Sincronización

Productor y consumidores en una cola de trabajos

\$4.4

CÓMPUTO PARALELO

`primos.c`

1. Un programa debe de computar el número primo #5000, el #6000 y el #7000 — tres cálculos independientes y lentos.
2. Calcularlos uno detrás del otro desperdicia los demás núcleos del procesador mientras el primero corre solo.
3. Ninguno necesita datos de los otros: es un caso ideal para repartir en hilos separados.
4. La idea: un hilo por cálculo con `pthread_create()`, y `pthread_join()` para juntar los tres resultados al final.

CÓMPUTO PARALELO

primos.c

```
// gcc -o primos primos.c -lpthread
#include <pthread.h>
#include <stdio.h>
void* compute_prime(void* arg) {
    int n = *((int*) arg), candidate = 2;
    while (1) {
        int factor, is_prime = 1;
        for (factor = 2; factor < candidate; ++factor)
            if (candidate % factor == 0) is_prime = 0;
        if (is_prime && --n == 0) return (void*) candidate;
        ++candidate;
    }
}
int main(void) {
    int which[3] = { 5000, 6000, 7000 }, i, prime;
    pthread_t t[3];
    for (i = 0; i < 3; i++)
        pthread_create(&t[i], NULL, &compute_prime, &which[i]);
    for (i = 0; i < 3; i++) {
        pthread_join(t[i], (void*) &prime);
        printf("Primo #%d = %d\n", which[i], prime);
    }
    return 0;
}
```

CANCELACIÓN SEGURA

tarea.c

1. Un hilo en segundo plano procesa un archivo grande usando un buffer temporal reservado con malloc().
2. El usuario debe poder cancelar esa tarea en cualquier momento — por ejemplo, si cierra la ventana.
3. Problema: si pthread_cancel() lo interrumpe a mitad de camino, ¿quién libera el buffer ya reservado?
4. Solución: pthread_cleanup_push()/pop() registran una función que libera el buffer automáticamente, se cancele el hilo o no.

Solución

tarea.c

```
// gcc -o tarea tarea.c -lpthread
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>
void liberar_buffer(void* buffer) {
    free(buffer);
}
void* procesar_archivo(void* arg) {
    void* buffer = malloc(4096);
    pthread_cleanup_push(liberar_buffer, buffer);
    while (1)
        pthread_testcancel(); // punto de cancelacion
    pthread_cleanup_pop(1);
    return NULL;
}
int main(void) {
    pthread_t tid;
    pthread_create(&tid, NULL, &procesar_archivo, NULL);
    sleep(2);
    pthread_cancel(tid);
    pthread_join(tid, NULL);
    return 0;
}
```

DATOS ESPECÍFICOS DE HILO

`registro.c`

1. Un programa reparte una tarea entre 5 hilos que ejecutan la misma función al mismo tiempo.
2. Por auditoría, cada hilo debe escribir su propio archivo de log — nunca uno compartido y entreverado entre hilos.
3. Todos corren el mismo código: necesitan una variable que valga distinto para cada hilo, no una global compartida.
4. Solución: una copia de la variable 'puntero al log' por cada hilo, indexada por una clave común.

DATOS ESPECÍFICOS DE HILO

registro.c

```
// gcc -o registro registro.c -lpthread
#include <pthread.h>
#include <stdio.h>
static pthread_key_t log_key;
void escribir_log(const char* msg) {
    fprintf((FILE*) pthread_getspecific(log_key), "%s\n", msg);
}
void cerrar_log(void* log) { fclose((FILE*) log); }
void* hilo_trabajador(void* arg) {
    char nombre[20];
    sprintf(nombre, "hilo%d.log", (int) pthread_self());
    pthread_setspecific(log_key, fopen(nombre, "w"));
    escribir_log("Iniciando tarea."); // ...hace su trabajo...
    return NULL;
}
int main(void) {
    pthread_t hilos[5]; int i;
    pthread_key_create(&log_key, cerrar_log);
    for (i = 0; i < 5; i++)
        pthread_create(&hilos[i], NULL, &hilo_trabajador, NULL);
    for (i = 0; i < 5; i++)
        pthread_join(hilos[i], NULL);
    return 0;
}
```

SINCRONIZACIÓN

`cola.c`

1. Varios hilos 'trabajadores' consumen tareas de una cola compartida; otra parte del programa las agrega a medida que llegan.
2. Sin protección, dos trabajadores pueden tomar el mismo trabajo a la vez, o leer la cola mientras otro la modifica.
3. Si la cola está vacía, tampoco conviene que los trabajadores la consulten todo el tiempo (busy-waiting).
4. Solución: un mutex protege la cola, y una variable de condición duerme a los trabajadores hasta que hay algo nuevo.

SINCRONIZACIÓN

cola.c

```
// gcc -o cola cola.c -lpthread
#include <pthread.h>
struct job { struct job* next; };
struct job* cola = NULL;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t hay_trabajo = PTHREAD_COND_INITIALIZER;
void encolar(struct job* j) {
    pthread_mutex_lock(&m);
    j->next = cola;
    cola = j;
    pthread_cond_signal(&hay_trabajo);
    pthread_mutex_unlock(&m);
}
void* trabajador(void* arg) {
    while (1) {
        pthread_mutex_lock(&m);
        while (cola == NULL)
            pthread_cond_wait(&hay_trabajo, &m);
        struct job* j = cola;
        cola = cola->next;
        pthread_mutex_unlock(&m);
        procesar(j); // fuera del mutex: no bloquea a los demas
    }
}
```